

daily_habit_monitoring.py

Separate PDF version of **daily_habit_monitoring.py** for the Daily Habit Monitoring System project.

```
import tkinter as tk
from tkinter import ttk, messagebox
import mysql.connector
from mysql.connector import Error
from datetime import date

MYSQL_PASSWORD = "YOUR_MYSQL_PASSWORD"

def connect_db():
    try:
        conn = mysql.connector.connect(
            host="localhost",
            user="root",
            password=MYSQL_PASSWORD
        )
        cursor = conn.cursor()
        cursor.execute("CREATE DATABASE IF NOT EXISTS habit_monitoring")
        cursor.close()
        conn.close()

        return mysql.connector.connect(
            host="localhost",
            user="root",
            password=MYSQL_PASSWORD,
            database="habit_monitoring"
        )
    except Error as e:
        messagebox.showerror("Database Error", str(e))
        return None

def create_tables():
    conn = connect_db()
    if conn:
        cursor = conn.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS users (
                user_id INT AUTO_INCREMENT PRIMARY KEY,
                username VARCHAR(50) UNIQUE NOT NULL,
                password VARCHAR(100) NOT NULL
            )
        """)
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS habits (
                habit_id INT AUTO_INCREMENT PRIMARY KEY,
                user_id INT NOT NULL,
                habit_name VARCHAR(100) NOT NULL,
                category VARCHAR(50),
                target VARCHAR(100),
                FOREIGN KEY (user_id) REFERENCES users(user_id)
                ON DELETE CASCADE
            )
        """)
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS habit_records (
                record_id INT AUTO_INCREMENT PRIMARY KEY,
                habit_id INT NOT NULL,
                record_date DATE NOT NULL,
                status VARCHAR(20) NOT NULL,
                FOREIGN KEY (habit_id) REFERENCES habits(habit_id)
                ON DELETE CASCADE
            )
        """)
        conn.commit()
        cursor.close()
        conn.close()

class HabitMonitoringSystem:
    def __init__(self, root):
        self.root = root
        self.root.title("Daily Habit Monitoring System")
        self.root.geometry("1000x650")
        self.root.resizable(False, False)
        self.current_user_id = None
        self.current_username = None
        create_tables()
        self.login_screen()

    def clear_screen(self):
        for widget in self.root.winfo_children():
            widget.destroy()

    def login_screen(self):
        self.clear_screen()
        frame = tk.Frame(self.root, padx=40, pady=30)
```

```

frame.pack(expand=True)

tk.Label(frame, text="DAILY HABIT MONITORING SYSTEM",
        font=("Arial", 22, "bold")).pack(pady=20)
tk.Label(frame, text="Login",
        font=("Arial", 18, "bold")).pack(pady=10)

tk.Label(frame, text="Username").pack(anchor="w")
self.login_username = tk.Entry(frame, width=35)
self.login_username.pack(pady=5)

tk.Label(frame, text="Password").pack(anchor="w")
self.login_password = tk.Entry(frame, width=35, show="*")
self.login_password.pack(pady=5)

tk.Button(frame, text="Login", width=25,
        command=self.login).pack(pady=15)
tk.Button(frame, text="Create New Account", width=25,
        command=self.registration_screen).pack()

def login(self):
    username = self.login_username.get().strip()
    password = self.login_password.get().strip()

    if username == "" or password == "":
        messagebox.showwarning("Warning",
                                "Please enter username and password.")
        return

    conn = connect_db()
    if conn:
        cursor = conn.cursor()
        cursor.execute("""
            SELECT user_id, username FROM users
            WHERE username=%s AND password=%s
            """, (username, password))
        result = cursor.fetchone()
        cursor.close()
        conn.close()

        if result:
            self.current_user_id = result[0]
            self.current_username = result[1]
            messagebox.showinfo("Success", "Login successful!")
            self.dashboard()
        else:
            messagebox.showerror("Login Failed",
                                "Invalid username or password.")

def registration_screen(self):
    self.clear_screen()
    frame = tk.Frame(self.root, padx=40, pady=30)
    frame.pack(expand=True)

    tk.Label(frame, text="CREATE ACCOUNT",
            font=("Arial", 22, "bold")).pack(pady=20)

    tk.Label(frame, text="Username").pack(anchor="w")
    self.reg_username = tk.Entry(frame, width=35)
    self.reg_username.pack(pady=5)

    tk.Label(frame, text="Password").pack(anchor="w")
    self.reg_password = tk.Entry(frame, width=35, show="*")
    self.reg_password.pack(pady=5)

    tk.Label(frame, text="Confirm Password").pack(anchor="w")
    self.reg_confirm = tk.Entry(frame, width=35, show="*")
    self.reg_confirm.pack(pady=5)

    tk.Button(frame, text="Register", width=25,
            command=self.register).pack(pady=15)
    tk.Button(frame, text="Back to Login", width=25,
            command=self.login_screen).pack()

def register(self):
    username = self.reg_username.get().strip()
    password = self.reg_password.get().strip()
    confirm = self.reg_confirm.get().strip()

    if username == "" or password == "":
        messagebox.showwarning("Warning", "All fields are required.")
        return

    if password != confirm:
        messagebox.showerror("Error", "Passwords do not match.")
        return

    conn = connect_db()
    if conn:
        try:
            cursor = conn.cursor()

```

```

        cursor.execute("""
            INSERT INTO users(username, password)
            VALUES(%s, %s)
            """, (username, password))
        conn.commit()
        cursor.close()
        conn.close()
        messagebox.showinfo("Success", "Registration successful!")
        self.login_screen()
    except Error:
        messagebox.showerror(
            "Error",
            "Username already exists or database error."
        )

def dashboard(self):
    self.clear_screen()
    header = tk.Frame(self.root, bg="#1f4e78", height=80)
    header.pack(fill="x")

    tk.Label(header, text="Daily Habit Monitoring System",
        font=("Arial", 22, "bold"),
        bg="#1f4e78", fg="white").pack(pady=10)
    tk.Label(header, text="Welcome, " + self.current_username,
        font=("Arial", 11),
        bg="#1f4e78", fg="white").pack()

    menu = tk.Frame(self.root, padx=30, pady=30)
    menu.pack()

    buttons = [
        ("Add Habit", self.add_habit),
        ("View Habits", self.view_habits),
        ("Search Habit", self.search_habit),
        ("Update Habit", self.update_habit),
        ("Delete Habit", self.delete_habit),
        ("Mark Completed", self.mark_completed),
        ("Daily Records", self.daily_records),
        ("Logout", self.logout)
    ]

    for i, (text, command) in enumerate(buttons):
        tk.Button(menu, text=text, width=25, height=2,
            command=command).grid(
            row=i // 2, column=i % 2, padx=15, pady=10
        )

def add_habit(self):
    self.clear_screen()
    frame = tk.Frame(self.root, padx=30, pady=20)
    frame.pack()

    tk.Label(frame, text="ADD NEW HABIT",
        font=("Arial", 20, "bold")).grid(
        row=0, column=0, colspan=2, pady=20
    )

    tk.Label(frame, text="Habit Name").grid(
        row=1, column=0, sticky="w", pady=8
    )
    self.habit_name = tk.Entry(frame, width=35)
    self.habit_name.grid(row=1, column=1)

    tk.Label(frame, text="Category").grid(
        row=2, column=0, sticky="w", pady=8
    )
    self.category = ttk.Combobox(
        frame,
        values=["Health", "Study", "Fitness", "Personal", "Work", "Other"],
        width=32,
        state="readonly"
    )
    self.category.grid(row=2, column=1)

    tk.Label(frame, text="Daily Target").grid(
        row=3, column=0, sticky="w", pady=8
    )
    self.target = tk.Entry(frame, width=35)
    self.target.grid(row=3, column=1)

    tk.Button(frame, text="Save Habit", width=20,
        command=self.save_habit).grid(
        row=4, column=0, colspan=2, pady=20
    )
    tk.Button(frame, text="Back", width=20,
        command=self.dashboard).grid(
        row=5, column=0, colspan=2
    )

def save_habit(self):
    name = self.habit_name.get().strip()

```

```

category = self.category.get()
target = self.target.get().strip()

if name == "" or category == "" or target == "":
    messagebox.showwarning("Warning", "Please fill all fields.")
    return

conn = connect_db()
if conn:
    cursor = conn.cursor()
    cursor.execute("""
        INSERT INTO habits
        (user_id, habit_name, category, target)
        VALUES(%s, %s, %s, %s)
        """, (self.current_user_id, name, category, target))
    conn.commit()
    cursor.close()
    conn.close()
    messagebox.showinfo("Success", "Habit added successfully.")
    self.dashboard()

def view_habits(self):
    self.clear_screen()
    tk.Label(self.root, text="MY HABITS",
             font=("Arial", 20, "bold")).pack(pady=15)

    columns = ("ID", "Habit Name", "Category", "Target")
    tree = ttk.Treeview(self.root, columns=columns,
                       show="headings", height=18)

    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=180)
    tree.pack(padx=20, pady=10)

    conn = connect_db()
    if conn:
        cursor = conn.cursor()
        cursor.execute("""
            SELECT habit_id, habit_name, category, target
            FROM habits WHERE user_id=%s ORDER BY habit_id
            """, (self.current_user_id,))
        for row in cursor.fetchall():
            tree.insert("", "end", values=row)
        cursor.close()
        conn.close()

    tk.Button(self.root, text="Back", width=20,
              command=self.dashboard).pack(pady=10)

def search_habit(self):
    self.clear_screen()
    tk.Label(self.root, text="SEARCH HABIT",
             font=("Arial", 20, "bold")).pack(pady=20)

    search_frame = tk.Frame(self.root)
    search_frame.pack()

    tk.Label(search_frame, text="Enter Habit Name:").pack(side="left")
    search_entry = tk.Entry(search_frame, width=30)
    search_entry.pack(side="left", padx=10)

    columns = ("ID", "Habit Name", "Category", "Target")
    tree = ttk.Treeview(self.root, columns=columns,
                       show="headings", height=15)

    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=180)
    tree.pack(pady=20)

    def search():
        for item in tree.get_children():
            tree.delete(item)

        keyword = search_entry.get().strip()
        conn = connect_db()
        if conn:
            cursor = conn.cursor()
            cursor.execute("""
                SELECT habit_id, habit_name, category, target
                FROM habits
                WHERE user_id=%s AND habit_name LIKE %s
                """, (self.current_user_id, "%" + keyword + "%"))

            for row in cursor.fetchall():
                tree.insert("", "end", values=row)
            cursor.close()
            conn.close()

    tk.Button(search_frame, text="Search",

```

```

        command=search).pack(side="left")
tk.Button(self.root, text="Back", width=20,
        command=self.dashboard).pack()

def update_habit(self):
    self.clear_screen()
    tk.Label(self.root, text="UPDATE HABIT",
        font=("Arial", 20, "bold")).pack(pady=15)

    form = tk.Frame(self.root)
    form.pack(pady=10)

    tk.Label(form, text="Habit ID").grid(row=0, column=0, pady=8)
    id_entry = tk.Entry(form, width=30)
    id_entry.grid(row=0, column=1)

    tk.Label(form, text="New Habit Name").grid(row=1, column=0, pady=8)
    name_entry = tk.Entry(form, width=30)
    name_entry.grid(row=1, column=1)

    tk.Label(form, text="New Category").grid(row=2, column=0, pady=8)
    category_box = ttk.Combobox(
        form,
        values=["Health", "Study", "Fitness", "Personal", "Work", "Other"],
        width=27
    )
    category_box.grid(row=2, column=1)

    tk.Label(form, text="New Target").grid(row=3, column=0, pady=8)
    target_entry = tk.Entry(form, width=30)
    target_entry.grid(row=3, column=1)

def update():
    habit_id = id_entry.get().strip()
    name = name_entry.get().strip()
    category = category_box.get().strip()
    target = target_entry.get().strip()

    if not habit_id or not name or not category or not target:
        messagebox.showwarning("Warning", "Please fill all fields.")
        return

    conn = connect_db()
    if conn:
        cursor = conn.cursor()
        cursor.execute("""
            UPDATE habits
            SET habit_name=%s, category=%s, target=%s
            WHERE habit_id=%s AND user_id=%s
            """, (name, category, target,
                habit_id, self.current_user_id))
        conn.commit()

        if cursor.rowcount > 0:
            messagebox.showinfo(
                "Success", "Habit updated successfully."
            )
            self.dashboard()
        else:
            messagebox.showerror("Error", "Habit not found.")

        cursor.close()
        conn.close()

    tk.Button(self.root, text="Update", width=20,
        command=update).pack(pady=15)
    tk.Button(self.root, text="Back", width=20,
        command=self.dashboard).pack()

def delete_habit(self):
    self.clear_screen()
    tk.Label(self.root, text="DELETE HABIT",
        font=("Arial", 20, "bold")).pack(pady=30)

    tk.Label(self.root, text="Enter Habit ID").pack()
    id_entry = tk.Entry(self.root, width=30)
    id_entry.pack(pady=10)

def delete():
    habit_id = id_entry.get().strip()
    if habit_id == "":
        messagebox.showwarning("Warning", "Enter Habit ID.")
        return

    answer = messagebox.askyesno(
        "Confirm",
        "Are you sure you want to delete this habit?"
    )
    if not answer:
        return
    conn = connect_db()

```

```

        if conn:
            cursor = conn.cursor()
            cursor.execute("""
                DELETE FROM habits
                WHERE habit_id=%s AND user_id=%s
            """, (habit_id, self.current_user_id))
            conn.commit()

            if cursor.rowcount > 0:
                messagebox.showinfo(
                    "Success", "Habit deleted successfully."
                )
                self.dashboard()
            else:
                messagebox.showerror("Error", "Habit not found.")

            cursor.close()
            conn.close()

tk.Button(self.root, text="Delete", width=20,
          command=delete).pack(pady=15)
tk.Button(self.root, text="Back", width=20,
          command=self.dashboard).pack()

def mark_completed(self):
    self.clear_screen()
    tk.Label(self.root, text="MARK HABIT COMPLETED",
            font=("Arial", 20, "bold")).pack(pady=20)

    form = tk.Frame(self.root)
    form.pack(pady=20)

    tk.Label(form, text="Select Habit").grid(
        row=0, column=0, padx=10
    )
    habit_box = ttk.Combobox(form, width=35, state="readonly")
    habit_box.grid(row=0, column=1)

    conn = connect_db()
    habit_data = {}

    if conn:
        cursor = conn.cursor()
        cursor.execute("""
            SELECT habit_id, habit_name FROM habits
            WHERE user_id=%s
        """, (self.current_user_id,))
        for habit_id, habit_name in cursor.fetchall():
            text = str(habit_id) + " - " + habit_name
            habit_data[text] = habit_id

        habit_box["values"] = list(habit_data.keys())
        cursor.close()
        conn.close()

    tk.Label(form, text="Status").grid(
        row=1, column=0, pady=15
    )
    status = tk.StringVar(value="Completed")

    tk.Radiobutton(
        form, text="Completed",
        variable=status, value="Completed"
    ).grid(row=1, column=1, sticky="w")

    tk.Radiobutton(
        form, text="Not Completed",
        variable=status, value="Not Completed"
    ).grid(row=2, column=1, sticky="w")

def save_record():
    selected = habit_box.get()
    if selected == "":
        messagebox.showwarning(
            "Warning", "Please select a habit."
        )
        return

    habit_id = habit_data[selected]
    conn = connect_db()

    if conn:
        cursor = conn.cursor()
        cursor.execute("""
            SELECT record_id FROM habit_records
            WHERE habit_id=%s AND record_date=%s
        """, (habit_id, date.today()))
        existing = cursor.fetchone()

        if existing:
            cursor.execute("""

```

```

        UPDATE habit_records SET status=%s
        WHERE record_id=%s
        """ , (status.get(), existing[0]))
    else:
        cursor.execute("""
            INSERT INTO habit_records
            (habit_id, record_date, status)
            VALUES(%s, %s, %s)
            """ , (habit_id, date.today(), status.get()))

    conn.commit()
    cursor.close()
    conn.close()

    messagebox.showinfo(
        "Success", "Today's habit status saved."
    )
    self.dashboard()

tk.Button(self.root, text="Save Status", width=20,
    command=save_record).pack(pady=15)
tk.Button(self.root, text="Back", width=20,
    command=self.dashboard).pack()

def daily_records(self):
    self.clear_screen()
    tk.Label(self.root, text="DAILY HABIT RECORDS",
        font=("Arial", 20, "bold")).pack(pady=15)

    columns = ("Record ID", "Habit", "Date", "Status")
    tree = ttk.Treeview(self.root, columns=columns,
        show="headings", height=18)

    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=200)
    tree.pack(padx=20, pady=10)

    conn = connect_db()
    if conn:
        cursor = conn.cursor()
        cursor.execute("""
            SELECT hr.record_id, h.habit_name,
            hr.record_date, hr.status
            FROM habit_records hr
            JOIN habits h ON hr.habit_id = h.habit_id
            WHERE h.user_id=%s
            ORDER BY hr.record_date DESC
            """ , (self.current_user_id,))

        for row in cursor.fetchall():
            tree.insert("", "end", values=row)
        cursor.close()
        conn.close()

    tk.Button(self.root, text="Back", width=20,
        command=self.dashboard).pack(pady=10)

def logout(self):
    self.current_user_id = None
    self.current_username = None
    self.login_screen()

if __name__ == "__main__":
    root = tk.Tk()
    app = HabitMonitoringSystem(root)
    root.mainloop()

```